

Fine-tuning MACE with the mdstats campaign CLI

This workflow has one purpose: turn correlated VASP trajectories into a MACE model that improves on LTA while retaining enough of the foundation model to remain stable. mdstats keeps data preparation, leakage control, replay, checkpoint selection, and verification in one resumable campaign.

Run every command from the package root:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml <command>
```

Storage retention

Use the unified storage command for STOR1-STOR5 accounting and retention. `storage report` is the read-only ownership and byte inventory (bare storage is an equivalent shorthand). STOR4 manual cleanup is explicitly tiered:

```
CAMPAIGN="python tools/mdstats-mlff-campaign.py --config campaign.toml"
$CAMPAIGN storage report
$CAMPAIGN storage cleanup --tier safe --dry-run
$CAMPAIGN storage cleanup --tier cache
$CAMPAIGN storage cleanup --tier recompute --apply
$CAMPAIGN storage cleanup --tier compact --apply
$CAMPAIGN storage cleanup --tier archive --dry-run
$CAMPAIGN storage cleanup --tier archive --apply
$CAMPAIGN storage archive create
$CAMPAIGN storage archive verify
$CAMPAIGN storage archive restore
$CAMPAIGN storage deduplicate
$CAMPAIGN storage deduplicate --apply
```

Every cleanup invocation prints and writes a capability plan first. `recompute` and `compact` require explicit `--apply`. The archive cleanup tier with `--apply` is additionally gated by STOR5: consequential hot bytes are removed only after a compressed archive is created, independently verified, and registered. `storage archive restore` reconstructs and re-verifies the exact hot layout. `storage deduplicate` is plan-only unless `--apply` is given and operates only on verified/frozen immutable campaign-owned files. External inputs, workspace production models, selected production raw checkpoints, protocol/selection/verification records, and diagnostic logs are never STOR4 deletion candidates.

For transition compatibility, the launcher still accepts the former top-level `cleanup`, `deduplicate`, and `archive` spellings, but they are intentionally omitted from top-level help. New scripts should use the `storage ...` hierarchy.

The visible campaign workspace stays compact:

```
mlff-campaign/
|-- campaign-manifest.json      # the one file you review and approve
|-- .mdstats/campaign.sqlite3  # durable internal state and provenance
|-- data/                      # generated MACE datasets/configurations
|-- runs/                      # checkpoints and per-run logs
|-- models/                   # qualified committee / verified production
`-- results/                  # benchmark, selection, and verification summaries
```

1. Create and edit the campaign

```
python tools/mdstats-mlff-campaign.py --config campaign.toml init
```

New 0.20.139 campaigns are generated with `checkpoint_strategy = "mlcv_nested_cv"`. Historical `adaptive_topk` remains supported only for campaigns whose original scientific authority is the pre-MLCV adaptive evaluator (or as the recorded transitional spelling of an already-prepared MLCV campaign).

Edit the five essential paths in `campaign.toml`:

- `training_root`: VASP XML trajectories;
- `foundation_model`: the exact MPA-0 checkpoint;
- `replay_train`: replay configurations used during training;
- `replay_monitor`: a disjoint replay split used by the training protocol;
- `replay_true_labels`: independent reference labels for the same replay geometries.

The normal MPA-0 workflow keeps `[replay].mode = "external_pseudolabel"`, so `replay_train` and `replay_monitor` stabilize training against labels generated by the exact foundation checkpoint. `replay_true_labels` is an independent evaluation input and does not change those training bytes. It may point to the original replay-preparation directory containing `mp_replay_selected.extxyz`; `mdstats` reconstructs the true-label train/monitor split from `replay_source_index`. It may instead contain already split files named `true_labels/replay_train.extxyz` and `true_labels/replay_monitor.extxyz` (the `replay_true_*` and `true_replay_*` filename variants are also accepted).

Change `[replay].mode` to `external_true_label` only when training itself should use the independent reference labels. The CLI binds pseudo-label replay to the foundation checkpoint byte hash and rejects unspecified provenance by default. Legacy configurations without `replay_true_labels` remain valid; their replay metric is diagnostic rather than an accuracy gate.

The default campaign now trains **multi-head replay fine-tuning only** with three optimizer seeds. Each seed receives three cross-validation folds plus one final-development fit, for 12 jobs total. This spends the default compute budget on the production-preferred replay-preserving method while measuring both fold and optimizer-seed variability. Naive/native target-only fine-tuning remains available under `[training.naive_fine_tuning]`, but it is disabled by default and should be enabled deliberately for comparison or ablation work.

Model precision

New campaigns expose exactly two learned-model precision modes:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml init --precision single
python tools/mdstats-mlff-campaign.py --config campaign.toml init --precision double
```

`single` is the default and keeps the MACE model FP32 through training, checkpoint monitor inference, evaluation, verification, committee inference, and export. `double` keeps the learned model FP64 through the same lifecycle. The former staged refine profile and a user-facing mixed model mode are retired for new campaigns; production commands fail closed on historical staged configurations rather than silently reinterpreting them.

This model-dtype choice does not make `mdstats` numerical analysis low precision. Reference fitting, PCA/SVD, geometry, metric/statistical reductions, observable analysis, and `mdstats`-owned persistent MD bookkeeping remain FP64 under either model mode. An FP32 checkpoint is never dtype-promoted merely to evaluate or export it as an FP64 model.

Acceleration backend

`init` now writes the correctness-first MH-1 default explicitly:

```
[acceleration]
backend = "e3nn"
only_cueq = false
require_available = true
```

The value is protocol-frozen; the campaign never silently changes backend later. CuEq remains available as an explicit optimization experiment through `--backend cueq` or by editing the configuration, but doctor must numerically qualify the exact source/training realization before preparation may proceed. Keep `only_cueq = false` for portable MACE checkpoints.

2. Check the machine and inputs

```
python tools/mdstats-mlff-campaign.py --config campaign.toml doctor
```

Do not continue until every blocking check passes. On the RTX 3090, confirm that CUDA is available and that the reported device is the expected GPU. For a CuEq campaign, require acceleration backend: `cueq` (real model smoke passed) and a listed cuequivariance, `cuequivariance-torch`, and CUDA-operations stack. The replay gate requires disjoint train/monitor geometries, explicit label provenance, coverage of all target elements, and the configured minimum counts. The bundled 33/6 smoke replay can test software but is intentionally too small for a default production campaign. Lowering the count gate with `allow_small_corpus = true` marks an exploratory workflow; it is not evidence of broad retention.

3. Prepare the data

First call:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare
```

The command writes a populated `campaign-manifest.json` and stops. It reads target temperature, thermostat, ensemble, timestep, and fixed-cell behavior from `vasprun.xml`. For the LTA profile it also treats filename strain tags as candidates, resolves compatible unstrained references, and promotes them only when the actual cell matrices reproduce the exact hydrostatic, orthorhombic, or symmetric right-polar shear definition. Thus `hydro+5` means +5%, while `hydro+0.05` retains fractional notation. A mismatch or ambiguity is warned and left conservatively ungrouped.

Review the inferred evidence, warnings, true replicas, trajectory continuations, and independent structural realizations. Routine XML facts should already be filled. Add a manual override only when source evidence is genuinely unresolved. A partial/truncated XML warning is retained for review. The later source gate accepts a trailing interrupted stream when complete controls, atom identities, and usable ionic records are recoverable; it records the shortened frame count and quality warning. Mid-file corruption or missing critical records still fails. Then approve the exact reviewed file:

```
python tools/mdstats-mlff-campaign.py prepare --approve-manifest
```

Approval is intentionally a fast transaction and returns after recording the reviewed manifest digest. It no longer starts the expensive source audit and DATA2-DATA9 preparation implicitly. Continue with a plain command:

```
python tools/mdstats-mlff-campaign.py prepare
```

For scripted legacy workflows only, the following performs both actions in one invocation:

```
python tools/mdstats-mlff-campaign.py prepare --approve-manifest --continue-after-approval
```

After source or tolerance changes, refresh the proposals before approval:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare --refresh-inferences
```

`prepare` is restartable. For current campaigns it rebuilds the target-size scientific substrate and nothing more: the source and canonical frame authority, the neutral statistical base and its protected split-exclusion relations, the target-size experiment definition, and the one common preparation shared by every candidate size.

prepare does not select a target size. It does not train a candidate, run the reducer, rank checkpoints, or build a per-size materialization. The candidate ladder it reports is a configured experiment definition, not a decision. Everything that decides N belongs to select-target-size.

The first prepare on an existing workspace also performs the one-time destructive target-size cutover. Retired selected-size, selector, role-domain, and coverage-selection records are quarantined rather than migrated: they are never reinterpreted as current authority, and no mixed old/new runtime exists. If the cutover is interrupted, re-run prepare to resume that exact cutover; the original process does not need to survive.

VASP sources are decoded once into a checksummed normalized frame cache, and subsequent stages reuse that cache. DATA4 is stored as checksummed content-addressed shards rather than a giant SQLite JSON value. Use status after an interruption; rerun the same command to resume.

The campaign prints per-source and per-stage progress with elapsed time and ETA. External MACE training, evaluation, and verification processes emit periodic heartbeats with elapsed time and log growth. Performance controls are optional:

```
[performance]
source_workers = 0           # automatic, bounded by CPU and memory
source_timeout_seconds = 0    # no per-source timeout
progress_interval_seconds = 30.0

[execution]
progress_interval_seconds = 60.0
training_progress_interval_seconds = 10.0
```

Use fewer source workers on memory-constrained machines. The production LTA benchmark used four workers and peaked near 2.7 GiB through DATA5.

Success signs:

- replay qualification passes with the expected label mode and foundation hash;
- the DATA5 leakage audit passes;
- all requested foundation frames finish;
- every qualified target-size/screening-seed variant materializes;
- the screening DATA9A gate reports passed with no blockers.

4. Run the bounded preflight

```
python tools/mdstats-mlff-campaign.py --config campaign.toml preflight
```

The command verifies the exact currently materialized DATA8 matrix and performs one real one-epoch MACE run under the same binary learned-model dtype and invariant FP64 scientific arithmetic policy used by the campaign, then reloads the target head and checks finite predictions. Preflight is operational evidence only: it never ranks target sizes or chooses checkpoints. The first TRAIN2 preflight binds the complete screening matrix and remains valid through unchanged $n1 \rightarrow n2 \rightarrow n3$ screening boundaries. After selected-size production materialization changes the DATA8 matrix, run the same preflight command again for that new matrix. When CuEq is selected, every generated YAML must contain `enable_cueq: true`, evaluation uses the same backend, and the training log must show MACE converting the model to CuEq. Use `preflight --check-only` only for a quick byte audit; it deliberately leaves the gate waiting and cannot authorize training. Run the full preflight before committing the RTX 3090 to a long campaign.

5. Select the target data size

```
python tools/mdstats-mlff-campaign.py --config campaign.toml select-target-size
```

This single restartable command is the only current screening entrypoint, and the only one that trains candidates or decides N . It trains the qualified candidate sizes to the exact configured coarse boundary n_1 , evaluates only those checkpoints and reduces the population, continues survivors to n_2 , then continues finalists to n_3 and freezes one N^* . Where the deterministic continuation schedule needs a total extent, it derives that value directly from terminal boundary n_3 ; it is not a second screening authority. The independent `[training].max_num_epochs` value n is reserved for a fresh selected-size production campaign and is not a fourth ordinary screening command. New campaigns default to screen $(n_1, n_2, n_3) = (1, 3, 10)$ and production $n = 30$. Epoch is a controlled variable here: an earlier checkpoint cannot substitute for the configured boundary even when it scores better.

Before scheduling anything new, the command reconciles the existing execution evidence, so re-running it after an interruption resumes from the authenticated current boundary instead of repeating completed work. Ordinary `train` and `evaluate` refuse to schedule the screen; they can never become a second screening scheduler.

The selected size and the exact selected data are never editable fields. They are re-derived from the authenticated terminal reducer state and the training order every time they are loaded, and any divergence between what is stored and what re-derives fails closed.

A successful final decision reports the frozen N . A typed scientific terminal outcome such as insufficient comparable candidates or nonconvergence at the configured ceiling is a **result**, not an interruption: it preserves its evidence, stays terminal on reload, and exposes no production next step. An incomplete screen with no terminal outcome stays operationally resumable instead.

6. Cross-validate the frozen training method

```
python tools/mdstats-mlff-campaign.py --config campaign.toml cross-validate
```

Cross-validation happens **after** N^* is frozen, and its universe is exactly the selected dataset T_{selected} . It answers a different question from target-size selection: not “how much data?”, which is already decided, but “is this training method sound on that data?”.

The plan is built from the canonical P_1 split-exclusion relation authority projected onto T_{selected} . Roles are allocated over whole relation components, so two frames that are non-separable - because they share a correlation unit, a geometry duplicate, a protected event window, or a replica/structural lineage - are never split across training and evaluation. Every eligible component is held out as outer evaluation exactly once across the K folds, and $K \geq 2$ is required: there is no zero-fold path that authorizes production without actual cross-validation. If the configured K cannot be satisfied on the selected data, the command fails before any training rather than quietly reducing the fold count.

Within a fold, order is the science. The fold trains fresh, its representative checkpoint is chosen on its own checkpoint monitor using **target metrics only**, and only then is the held-out fold evaluated. Replay evidence can make a checkpoint inadmissible through the `TRAIN2` admissibility policy, but it earns no score bonus, weight, or tie-break.

Acceptance is exact rather than aggregate: every required fold of every required CV seed must satisfy the configured target-only predicate. A good mean cannot carry a failing fold, a missing fold is not a pass, and cross-fold dispersion is recorded as a diagnostic without gating. A cross-validation failure is a *methodological* result: it never selects another N , reruns the target-size screen, or reinterprets the frozen selection.

Configure it under `[post_selection.cv]`. Note that `cross_validation_folds` and `fold_partition_seed` under `[training.*]` belong to the retired pre-target lifecycle and are not this authority.

6b. Train the fresh final production model

```
python tools/mdstats-mlff-campaign.py --config campaign.toml train-production
```

Final production is new training on the **full exact** T_{selected} under precisely the method that cross-validation accepted - not a promotion of whichever fold or screening run happened to score well. Each configured production seed starts from the canonical initialization with a fresh optimizer and fresh RNG state, and screening or fold trajectories are not admissible parents. Screen, CV-fold, and production runs keep disjoint run identities, checkpoint roots, and restart ownership even when their N and numeric seed coincide.

The production epoch horizon is `[training].max_num_epochs`. It is independent of the screening ladder's n_3 and of the cross-validation budget, so raising it invalidates only production descendants: the frozen selection and the accepted cross-validation evidence both remain current, and no CV rerun is required. Final model selection uses the frozen M3 development reserve, which is inherited P2 evidence rather than a production setting; locked and calibration evidence remain strictly downstream and cannot influence checkpoint, seed, or horizon choice.

Both commands re-establish the current selection through the canonical loader before doing anything, and publish their results under a commit-time currentness fence. Work that began under an older canonical generation stays available as historical evidence but can never become current after prepare has committed a newer one.

`materialize` remains **unavailable**: post-selection materialization is not a standalone step any more. Each of the two commands above materializes exactly the fold-local or full- T_{selected} workload that its own authenticated plan authorizes.

7. Inspect, run, and resume production training

```
python tools/mdstats-mlff-campaign.py --config campaign.toml train --dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml train
```

For TRAIN2, public `train` is now production-only: it requires frozen N^* , the selected-size final/CV DATA8 matrix, and a preflight receipt bound to that exact matrix. Target-size screening cannot be driven through public `train`; it is owned by `select-target-size`. The dry run lists exact fold/final jobs. The real command supervises them, preserves logs and every checkpoint, and resumes failed or interrupted attempts with the qualified binary model-precision policy.

CUDA training uses adaptive process-level concurrency by default. The scheduler checks the active CPU affinity/cgroup quota, available host RAM, and aggregate GPU state before launch. It starts with exactly one independent MACE job. A new job is considered only after every active process has produced fresh optimizer records and remained in true epoch compute for the full averaging window. GPU utilization and VRAM naturally fluctuate, so the scheduler averages the complete window rather than waiting for variance to disappear. The projected mean aggregate VRAM and GPU utilization after adding one job must both remain strictly below their configured ceilings. Both defaults are 90%. After each promotion, calibration restarts; initialization and validation of the new process cannot authorize another promotion. If a post-add average reaches a ceiling, active jobs continue but the future replacement target is reduced by one. One job failure stops admission of queued jobs by default.

Relevant runtime-only controls are under `[execution]`:

```
training_progress_interval_seconds = 10.0
parallel_training_jobs = 0
minimum_parallel_training_jobs = 1
maximum_parallel_training_jobs = 4
training_gpu_memory_fraction = 0.90
training_gpu_utilization_fraction = 0.90
estimated_training_vram_mib_per_job = 6144.0
estimated_training_ram_mib_per_job = 8192.0
parallel_training_epoch_stabilization_seconds = 60.0
parallel_training_epoch_activity_timeout_seconds = 120.0
parallel_training_epoch_stability_samples = 12
# Compatibility fields accepted by older configs; no longer used as variance gates.
```



```
parallel_training_stability_relative_tolerance = 0.10
parallel_training_utilization_stability_absolute_tolerance = 8.0
parallel_training_memory_growth_margin = 1.05
parallel_training_utilization_growth_margin = 1.05
parallel_training_monitor_interval_seconds = 10.0
stop_scheduling_after_failure = true
```

These settings affect runtime scheduling only; they do not change DATA8 bytes or the frozen optimizer protocol. To retry one job:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml train --run-id RUN_ID
```

Beginning in 0.20.132a0, MLCV-MON1 replaces the common target monitor with role-correct run-local monitors before training:

- each CV fold uses `V_i_light`, a deterministic subset (up to 256 configurations) of that fold’s nested training-side checkpoint-selection domain `V_i_full`; the rotating outer CV fold remains untouched;
- each final-development run uses `D_light`, a deterministic subset (up to 256 configurations) of complete held-out validation `D_full`;
- replay uses `R_light`, a deterministic subset (up to 512 configurations) of complete independent TRUE_DFT replay validation `R_full`; and
- every run receives a deterministic target-training diagnostic subset (up to 256 configurations) from its exact gradient-training membership.

MACE evaluates the training diagnostic through the target head under the distinct `target_train_diagnostic` log label. It is placed before ordinary validation loaders and is selection-inert, so the ordinary target validation remains MACE’s native checkpoint/patience scalar. `pt_train_file` remains the configured replay-training corpus (including pseudo labels when selected). These monitor artifacts never supply gradients. Monitor sizes and `online_monitor_seed` are immutable policy identity.

Beginning in 0.20.140a0, MLCV replay retention is foundation-relative. The absolute target criterion remains `T_full_max`, while `DeltaR_max=(target_weight/replay_weight)*T_full_max` is a replay **degradation** budget. `mdstats` freezes matched `R0_light` and `R0_full`; per-epoch STOP1 uses `DeltaR_light=R_light-R0_light`, and SELECT1 uses `DeltaR_full=R_full-R0_full`. The default control geometry is `T_stop=f_T*T_full_max` and `DeltaR_stop=f_R*DeltaR_max` with configurable `f_T=0.80`, `f_R=1.20`, and a three-epoch minimum floor. At 30 meV/A and 1:1, those are 24 meV/A target and 36 meV/A replay **degradation**, not raw replay RMSE. Foundation evaluation establishes zero and is never rejected merely because its raw replay RMSE exceeds 30 meV/A. RANK1/SELECT1/AGG1/FINAL1 score signed replay degradation; raw replay RMSE remains diagnostic evidence.

MLCV-RANK1 is implemented in 0.20.134a0. Every checkpoint with complete finite lightweight target/replay RMSE is rankable, including values outside the future full-validation thresholds. Each training run retains at most five candidates independently; shorter runs retain exactly the available candidates without duplication. The default 1:1 lightweight score is the arithmetic mean of target and replay force RMSE, with deterministic ties preferring lower target RMSE, then lower replay RMSE, then earlier epoch and checkpoint SHA. Ranking launches no new inference.

MLCV-SELECT1 is implemented in 0.20.135a0. Every retained candidate of each completed run is fully evaluated before that run chooses a representative. Fold runs use complete nested `V_i_full` plus complete TRUE_DFT `R_full`; the untouched outer fold is not queried. Final-development runs use complete held-out `D_full` plus `R_full`. The 30 meV/A target criterion and weight-derived replay criterion are component-wise hard gates here, together with any configured energy/focus/stress/worst-condition limits. Only survivors are ranked by the full weighted score, producing exactly one run representative or an explicit `no_representative` result.

MLCV-AGG1 is implemented in 0.20.136a0. After each fold representative is frozen, the exact checkpoint is evaluated once on the complete untouched outer fold. The target outer-fold force RMSE must satisfy the configured target ceiling; replay is not rotated or re-inferred, and the representative's authenticated complete TRUE_DFT R_full error from SELECT1 is reused for replay and combined-score reporting. For each seed, target/replay/combined summaries include mean, sample standard deviation, minimum, maximum, range, and worst fold. All configured folds must survive. Dispersion is diagnostic-only, and fold representatives are permanently ineligible for production export.

MLCV-FINAL1 is implemented in 0.20.137a0. Configured-CV failure blocks production comparison at the recipe level. Otherwise, only qualified full-development representatives are compared, all on identical D_full + R_full evidence under the same SELECT1 policy. The deterministic best seed becomes the single production verification candidate, while every qualified final seed is exported separately as a target-head committee member; failed final seeds are omitted rather than padded. Fold representatives can never enter either path. FINAL1 does not publish the verified production model or activate locked post-freeze evidence; those actions remain owned by MLCV-VERIFY1. Generated method tables use seed_mode = "optimizer_only"; optional optimizer_and_cv_partition varies deterministic CV partitions per optimizer seed for broader robustness sampling without changing final full-development training membership.

Extend a completed MLCV campaign by one optimizer seed

A completed conventional-MLCV campaign can be enlarged before verify freezes production authority. For example, to add seed 4 to the default three-seed multihead_replay, n512, three-fold campaign:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml extend-seed --seed 4
```

The command is intentionally seed-only. It requires seed_mode = "optimizer_only", so the appended realization uses the exact same DATA5/MLCV fold partition as the existing seeds. Before launching MACE, mdstats compares the new DATA8 MlcvRoleCatalog digest with the parent seeds and fails closed if the fold roles differ.

The operation is resumable and does not retrain or re-evaluate the parent seeds. It appends the new seed to the method's TOML seed array, reuses DATA2-DATA6 plus the exact promoted DATA7 feature-fit archives and existing DATA8 variants, materializes only the new seed variant, validates/preflights the expanded DATA8 matrix, trains only the new seed's K fold jobs plus its final-development job, reuses authenticated parent SELECT1 and outer-fold records, then rebuilds campaign-level AGG1/FINAL1 over the strict seed superset. Promoted DATA7 reuse remains available even if post-training cleanup removed the transient shared DATA7 cache. The new final model becomes an additional committee member only if it passes the same CV, target, and replay-retention gates.

Preview without changing state:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml extend-seed --seed 4 --dry-run
```

For campaigns with more than one method or selection size, specify --training-mode and/or --selection-size. Seed extension is refused after VERIFY1, locked-E evaluation, production publication, or protocol freeze; those authorities require a new campaign identity rather than retroactive committee mutation.

train also accepts --training-mode, --seed, and --selection-size filters, which is how the extension scheduler isolates the newly appended jobs.

8. Evaluate the selected production trajectories

```
python tools/mdstats-mlff-campaign.py --config campaign.toml evaluate
```

For current TRAIN2 campaigns, public evaluate is available only after selected-size production training. Unlike the target-size study, checkpoint epoch is now selectable: an earlier admissible checkpoint may win over the

full-horizon n checkpoint according to the frozen production checkpoint-selection policy. Production evaluation cannot modify the frozen target-size authority.

For historical conventional MLCV campaigns, `evaluate` executes MLCV-SELECT1, then MLCV-AGG1, then MLCV-FINAL1 once the full campaign is complete. Each run first fully evaluates all retained top-five candidates on its correct complete checkpoint-selection domains and freezes one representative or an explicit failure. AGG1 then evaluates each frozen fold representative exactly once on its untouched outer CV target fold. The outer result can pass or fail that fold but cannot choose another epoch. Per-seed target/replay/combined CV statistics are written separately; all configured folds must survive, while cross-fold dispersion remains diagnostic-only. FINAL1 then compares only qualified full-development representatives, freezes one best seed as the production verification candidate, and exports all qualified final seeds as the active-learning committee. It does not publish the verified production model. Historical pre-MLCV adaptive campaigns retain their original ADAPT-EVAL1 behavior and bounded, exhaustive, and `multi_fidelity` remain available for compatible old campaigns.

The 256-frame target and 512-frame replay monitors remain lightweight training evidence; they are not relabeled as full evaluation. Naive fine-tuning still receives no replay gradients, but during training the fixed true-replay monitor is evaluated through its target head as a validation-only loader, making its STOP1/RANK1 score directly comparable to multi-head replay.

With `replay_true_labels` configured, each purchased finalist and the frozen foundation are evaluated on the independent true-label replay domain. This preserves pseudo-label replay for gradient training while making final replay accuracy authoritative. A finalist must pass the the absolute target force-RMSE ceiling plus the replay-degradation budget and retained energy, mobile-ion force, stress, and worst-condition safety gates before its weighted full score can be ranked. Foundation-relative replay degradation is reported as a diagnostic rather than the default hard selector.

MLCV-VERIFY1 is implemented in 0.20.138a0. The `verify` command visits only qualified FINAL1 full-development representatives, in FINAL1 score order. With `fallback_to_next_qualified_final_seed = true` (default), bounded-NVE failure may advance to the next qualified final seed; fold models can never enter this fallback. The first physical passer is frozen before locked data are exposed. Only then is sealed target test E materialized and evaluated target-only on the byte-identical frozen target-head model. Locked E cannot select another seed/checkpoint: failure is terminal campaign/review evidence under the current identity. `models/production_best.model` is atomically published only after the frozen physical passer also passes locked E; the qualified seed committee remains separate for active learning.

MLCV-MIGRATE1 was introduced in 0.20.139a0; 0.20.140a0 advances its lifecycle schema for replay-degradation authority. New campaigns use `checkpoint_strategy = "mlcv_nested_cv"`, frozen against immutable campaign plus ROLE1/MON1 identities. Historical absolute-replay records remain readable and are preserved under their original digests. Transitional 0.20.131a0-0.20.139a0 replay-dependent STOP1/RANK1/SELECT1/AGG1/FINAL1 evidence is stale under the new semantics and is not silently reranked or algebraically converted. Current `train` refuses historical MLCV DATA8 stop policy authority and instructs regeneration under 0.20.140a0; an old lifecycle record is archived by content digest before the current lifecycle is installed. VERIFY1/locked-E work may be reused only when the corrected FINAL1 freezes the exact same production checkpoint.

For **historical pre-MLCV campaigns only**, ADAPT-VERIFY1 (introduced in 0.20.127a0) consumes ordered fully admissible EVAL1 evidence. It verifies the lowest-full-score candidate first and, if a hard bounded-NVE gate fails, tries the next already fully evaluated admissible candidate when `fallback_to_next_full_evaluation_candidate = true`. No additional target/replay evaluation is purchased during fallback. New MLCV campaigns do not enter this historical verification path after AGG1; MLCV-FINAL1 now owns production comparison, and MLCV-VERIFY1 owns physical verification and verified production publication.

Evaluation records are bound to the monitor artifact digests, model hashes, and evaluation-policy digest. Adding or changing `replay_true_labels` therefore invalidates stale pseudo-label records and reruns only the affected checkpoint inference. If a completed campaign already pruned unselected checkpoints, `mdstats` refreshes the retained selected checkpoint and records that reduced scope rather than pretending deleted checkpoint bytes were re-evaluated.

An interrupted campaign can be evaluated without finishing unrelated methods or seeds. To inspect one completed group explicitly:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml evaluate \
  --training-mode multihead_replay --seed 1
```

The evidence hierarchy is:

- all configured folds plus final complete: complete-variant interim evidence;
- at least two folds complete: reduced cross-fold evidence with missing-fold and missing-final warnings;
- one fold or one final model complete: checkpoint-monitor and bounded-stability evidence only, with an explicit no-cross-validation warning.

`--selection-size` can narrow the group further. `--require-complete` refuses interim evaluation. Interim evaluation exports only the completed selected models, does not prune checkpoints, and does not create a production freeze. If additional models finish after the interim selection, rerun `evaluate` before `verify` so the verification scope is current.

A convincing result has:

- target force error clearly below raw MPA-0;
- Li, Na, and K errors comparable to the global error;
- no isolated composition, temperature, or strain failure;
- replay degradation below the configured limit;
- similar conclusions across folds and seeds;
- a learning curve that is flattening rather than still improving sharply.

The compact result is `results/fine-tuning-result.json`.

ADAPT-MIGRATE1 is active in 0.20.128a0 and closes the adaptive revision. Immutable run protocol identity now prevents an adaptive campaign from being switched back to historical bounded|exhaustive|multi_fidelity evaluation by editing TOML, and prevents a historical campaign from being reinterpreted as `adaptive_topk` without a new scientific campaign identity. A schema-neutral protocol-freeze authority record lets storage/restart code validate either historical committee freezes or adaptive deployment freezes without changing the original scientific evidence. Completed 0.20.127 adaptive campaigns are reconciled by rerunning `verify` once: model inference, full evaluation, and NVE are reused; only the generic freeze-authority alias and immutable migration receipt are added. Once an adaptive deployment is frozen, later `evaluate` invocations only reuse the frozen EVAL1 authority and do not create a second selection history. `storage report` exposes freeze authority and preserved historical evaluator evidence read-only, while STOR cleanup/deletion boundaries are unchanged.

9. Verify bounded MD stability

Add equilibrated verification structures to `[verification].structures`, then:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml verify
```

For a completed `adaptive_topk` campaign, verification is sequential rather than committee-wide. The best fully admissible full-evaluation candidate receives the complete configured structure x temperature NVE matrix. If it fails a hard finite/drift/minimum-distance/maximum-force threshold, `mdstats` falls back to the next admissible full-

score candidate. Verification stops at the first pass, and only that exact target-head artifact is published in `models/`. A fold-run candidate is allowed to win; `mdstats` does not fabricate a final-development committee around it. The learned-model dtype is unchanged (single -> FP32, double -> FP64), while MD state and analysis remain FP64.

For an interrupted campaign, the historical interim verification path is still available for the latest completed-model set. It remains explicitly weaker evidence and cannot create an adaptive production deployment freeze. Use `verify --require-frozen` when you want to prohibit interim checks.

The hard total-energy-drift limit remains 26 meV/atom/ps. A good model should be comfortably below it, not merely just under it. Interim output is written to `results/interim-completed-model-verification.json`; full output remains `results/production-verification.json`. Neither result replaces matched RDF, coordination, site occupancy, VDOS/VACF, strain response, or sufficiently long diffusion comparisons. Interim success does not authorize deployment.

When the result is not good

Energy bad, forces reasonable: inspect the DFT energy channel and residual atomic reference (E0) fit. More epochs usually do not repair a reference offset.

Training good, monitor poor: the data are too correlated, the learning rate is too high, or the model is overfitting. Add independent configurations, reduce the learning rate, or stop earlier.

Li/Na/K forces poor: add ion-site, transition, window-crossing, and coordination-change configurations. Global framework-dominated RMSE can hide this.

Replay retention poor: enlarge and diversify replay, reduce the target-head weight or learning rate, and compare with the naive baseline.

MD unstable or atoms collapse: add short-range repulsive, strained, and high-restoring-force configurations. Reject the current checkpoint.

Only NVE drift is poor: reduce the timestep first, confirm critical-FP64 is active, and compare FP32 with a bounded FP64 run.

Editing `campaign.toml` after a completed stage makes that stage visibly stale. Rerun the reported stage so new choices are rebound to the campaign evidence; the CLI will not silently reuse an authorization issued for different configuration bytes.

At any point:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml status
```

It shows the current gate, its reason, and the next safe command.

Large DATA6/DATA7 artifact policy

`model.artifact_shard_size` controls immutable DATA6 descriptor/prediction shard size. The generated production value is 128 frames. A smaller value limits the work lost from an interrupted active shard; a larger value further reduces filesystem metadata operations but increases temporary shard memory and recovery granularity. Completed shard records are journaled only after the shard is durable. Existing legacy per-frame sidecars are verified and reused.

Descriptor shards persist the summary features consumed by DATA7. DATA7 therefore does not reload atomic descriptor tensors unless an older sidecar lacks summaries. Prediction shards support scalar-only energy access so residual atomic-reference fitting does not load force or stress arrays.

DATA7 archives store large numerical members as uncompressed native NumPy members. The fitted frame matrix is restored as a read-only memory map directly from the archive, while centers, scales, and projections remain native arrays. Do not modify a mapped archive in place; campaign artifacts are immutable and checksum-bound.

`evaluation.batch_size` controls candidate checkpoint batches. CUDA out-of-memory errors reduce the active batch recursively. `cache_monitor_datasets = true` reuses only parses bound to unchanged path/stat identity and the expected SHA-256. `cache_replay_baseline = true` similarly reuses foundation replay metrics only when model, monitor, head, and policy identities all match.

Checkpoint evaluation and bounded NVE verification use adaptive independent-job concurrency, but CUDA admission now uses a deliberately long one-job calibration rather than repeated short windows. Evaluation builds one campaign-wide queue across runs and checkpoints, so the calibration can continue across successive short jobs. CUDA starts with exactly one job and remains at concurrency one for 300 seconds by default. Sampling starts at task launch and therefore spans checkpoint/model setup, monitor work, device transfer, inference, NVE integration, and other stage changes.

GPU utilization and incremental VRAM are measured relative to the pre-launch GPU baseline. Samples below 1% activity are discarded independently for the two metrics. The remaining samples are sorted independently. `mdstats` discards the highest 5% as transient peaks, then averages the next-highest 10% of retained GPU-utilization values and, independently, the next-highest 10% of incremental-VRAM values. With the defaults this is approximately the 85th–95th percentile band. These robust upper-band means become the fixed per-job estimate for the rest of the queue. `mdstats` then jumps directly to the largest projected concurrency whose aggregate GPU utilization and VRAM remain strictly below 90%, additionally bounded by CPU, RAM, explicit job caps, and task count. If a short calibration job finishes, the next queued job continues serially under the same 300-second clock. The scheduler does not wait unnecessarily if the queue finishes before calibration is complete. The 90% ceilings are soft parallel-expansion envelopes, not single-job execution proof: a successfully completed one-slot calibration proves that serial execution is viable, so measured demand above a soft envelope caps concurrency at one (serial fallback) instead of blocking the queue. Only actual execution failure, such as a genuine CUDA out-of-memory error, or device unavailability terminates queued work. If preflight GPU telemetry is unavailable while the CUDA device is present, `mdstats` starts in conservative serial mode without inventing expansion headroom. After calibration, the calibrated GPU-utilization estimate is frozen: instantaneous GPU-utilization spikes do not ratchet concurrency downward. Live telemetry retains only the hard VRAM guard, because actual memory saturation can cause OOM; that guard throttles additional launches while active jobs occupy the target but never reduces an idle queue below one launchable job.

The CLI continues to print per-task stage transitions such as authenticating checkpoint artifact, reconstructing deployable MACE model, loading target monitor, evaluating candidate on LTA target monitor, loading verification MACE model, and running bounded NVE trajectory. During calibration, scheduler heartbeats also report elapsed calibration time plus retained nonzero GPU/VRAM sample counts. Monitor parsing and foundation baseline evaluation remain synchronized so concurrent candidates reuse one authenticated parse and one foundation result. Admitted CUDA jobs use separate PyTorch streams, and verification cases never share a mutable ASE/MACE calculator.

CPU evaluation/verification retains a 20-second workload window with a projected 90% host-utilization ceiling. Training remains separate and retains its 60-second true-epoch calibration. RAM remains capped at 80%.

Shared runtime controls are under `[execution]`; phase-specific keys with `evaluation_` or `verification_` prefixes may override them:

```
parallel_inference_jobs = 0
maximum_parallel_inference_jobs = 0
inference_cpu_utilization_fraction = 0.90
```

```

inference_gpu_memory_fraction = 0.90
inference_gpu_utilization_fraction = 0.90
inference_estimated_vram_mib_per_job = 4096.0
inference_estimated_ram_mib_per_job = 4096.0
parallel_inference_calibration_window_seconds = 300.0
parallel_inference_cpu_calibration_window_seconds = 20.0
inference_gpu_minimum_activity_fraction = 0.01
inference_gpu_calibration_peak_trim_fraction = 0.05
inference_gpu_calibration_band_fraction = 0.10
parallel_inference_stability_samples = 3
inference_memory_growth_margin = 1.05
inference_utilization_growth_margin = 1.05
parallel_inference_monitor_interval_seconds = 2.0

```

The exact shared `inference_gpu_calibration_peak_trim_fraction = 0.10` generated by 0.20.91a0 migrates to the current 0.05 default. Explicit phase-specific peak-trim values and other custom shared values remain unchanged.

The following deprecated stabilization keys remain readable:

- `parallel_inference_stabilization_seconds`
- `parallel_evaluation_stabilization_seconds`
- `parallel_verification_stabilization_seconds`

Exact shared generated defaults from 0.20.86a0 (10.0), 0.20.87a0 (60.0), 0.20.88a0 (20.0), and 0.20.89a0 (180.0) migrate to the current 300-second CUDA calibration. Explicit phase-specific values and other custom shared values remain unchanged. The 4096 MiB per-job VRAM setting is no longer a hard pre-calibration cap; it is a fallback only if no incremental-VRAM sample crosses the configured activity floor.

For example, `parallel_evaluation_jobs = 2` caps checkpoint evaluation at two concurrent jobs, while `verification_gpu_minimum_activity_fraction = 0.02` changes only verification's near-zero filter. `verification_gpu_calibration_peak_trim_fraction = 0.10` and `verification_gpu_calibration_band_fraction = 0.15` discard the highest 10% and average the next 15% for verification. The historical `verification_gpu_calibration_upper_tail_fraction` key remains accepted as an alias for the band width. These controls are runtime-only and do not alter checkpoint, metric, selection, or verification-case scientific identity.

Automatic CPU, RAM, and GPU acceleration

The default TOML uses zero-valued worker and batch fields to request automatic planning. `mdstats` detects the effective CPU allocation, currently available RAM, selected CUDA device, and free VRAM. CPU and GPU/VRAM defaults are 90%; RAM alone remains 80%. It prints the plan before each expensive stage. A 128-thread host normally budgets 115 threads, but a 27-run campaign can use at most 27 trajectory workers. Memory estimates may reduce that count further.

`source_workers`, `feature_workers`, and `lta_workers` may be set explicitly, but positive values are still clipped by CPU and RAM bounds. `inference_batch_size = 0` selects a VRAM-bounded DATA6 batch and halves it after CUDA OOM. `num_workers = 0` selects an economical MACE DataLoader pool. The GPU is used for MACE/cuEq numerical kernels; XML and catalog stages remain isolated CPU processes.

Revision-4 process and graph caches

Two optional environment variables bound process-local reuse during long checkpoint and feature campaigns:

```

export MDSTATS_MLFF_MONITOR_CACHE_BYTES=$((512 * 1024 * 1024))
export MDSTATS_MACE_GRAPH_CACHE_BYTES=$((512 * 1024 * 1024))

```

Set either value to 0 to disable that cache. Monitor entries are authenticated by path, size, modification time, inode, and expected SHA-256. MACE graph entries are CPU-resident, policy-keyed, and reused only while the exact same immutable ASE objects remain live. These caches do not change model outputs or campaign lineage.

Replay metrics during checkpoint evaluation

`external_pseudolabel` replay does not provide independent accuracy labels. When no `replay_true_labels` directory is configured, candidate force RMSE against pseudo labels is reported only as absolute candidate-versus-foundation disagreement; checkpoint admissibility and ranking use the DFT-labeled target monitor. When the independent true-label directory is present, evaluation automatically substitutes its geometry-identical monitor and applies the configured relative replay-degradation threshold to foundation and candidate errors against the same true references. Training still uses pseudo labels unless `[replay].mode` is changed explicitly.

Phase-separated acceleration default (0.20.193a0)

Plain `init` now generates `source/DATA6/evaluation backend = "e3nn"` and `TRAIN2 training_backend = "cueq"`. CuEq is training-only by default; `only_cueq=false` converts saved models back to portable e3nn checkpoints. `doctor` must qualify both realizations and fails instead of silently falling back. Existing campaign TOML without `training_backend` keeps its historical unified backend. Use `init --training-backend e3nn` for a reference all-e3nn training campaign.